Final Project Report

VLSIntelligence Group

Part 1

This part implements the mapping of a 4-bit quantized VGGNet layer onto an 8×8 weight-stationary systolic array.

1.1 Train model

We trained a VGGNet model with 4-bit input activations and 4-bit weights, and to facilitate hardware mapping, we targeted the 27th convolution layer and "squeezed" its dimensions by reducing both input and output channels to 8. We achieved a baseline accuracy of 92.29%. The hardware quantization error (`psum_recovered`) was calculated by comparing the simulated hardware output against the pre-hooked input of the subsequent layer. The resulting error is approximately 2.46×10^{-7}.

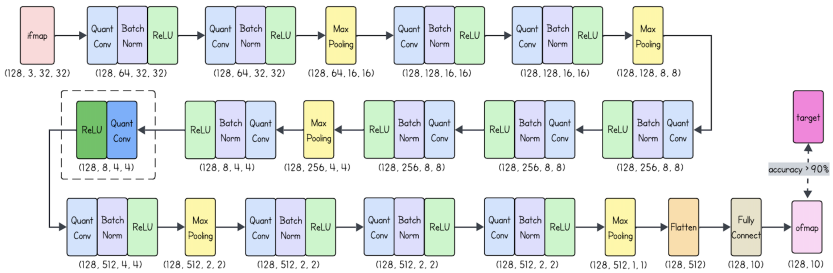


Figure 1: Compressed VGGNet Architecture

1.2 Hardware Architecture

The RTL design is hierarchically structured into a `corelet` and a top-level `core` module. `corelet` encapsulates the compute logic, including the (1) L0 buffer for loading weights and streaming activations, (2) 8×8 Weight-Stationary MAC array (4-bit), (3) Output FIFO (OFIFO) for collecting partial sums, and (4) Special Function Unit (SFU) for accumulation and ReLU operations. And `core` integrating the `corelet` with the Scratchpad Memory subsystem, comprising Input SRAMs and Psum SRAMs. Detailed component specifications are provided in Appendix A.2.

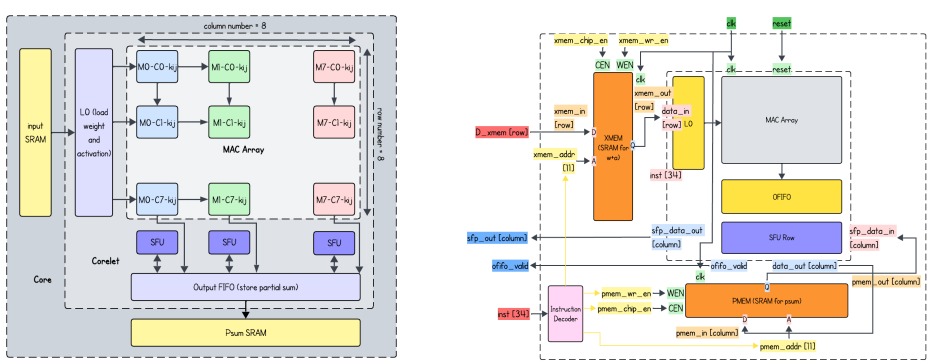


Figure 2: Core Architecture and Signal Interface

1.3 Testbench and Verification

The testbench functions as the main controller, driving the top-level `core` module through the complete execution lifecycle. The verification sequence executed the following stages:

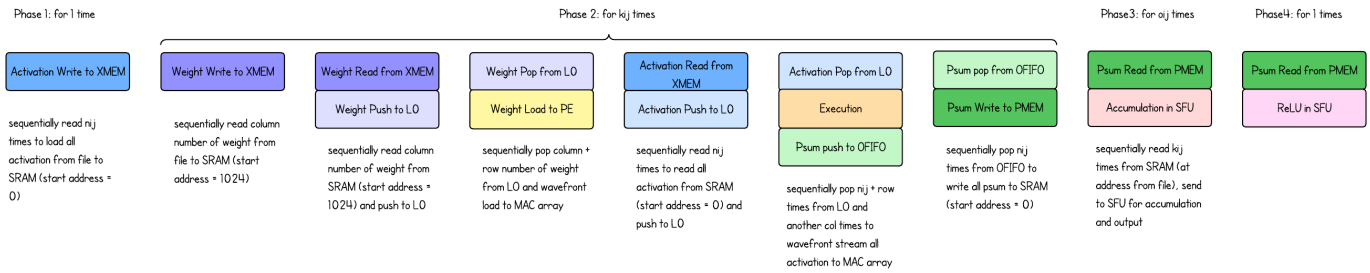


Figure 3: Systolic Array Execution Process

1.4 FPGA Synthesis Result

We synthesized the design targeting the Cyclone IV GX FPGA using Intel Quartus. Following the complete synthesis process, performance metrics were analyzed under the specified Timing and Power conditions. The performance results for the Vanilla PE implementation are summarized below:

Configuration	Max Freq (MHz)	Dyn Power (mW)	GOPS	GOPS/W
Vanilla Version	127.89	31.7	16.37	516.4

Part 2

This part reconfigures the Processing Elements (PEs) and Core architecture into a dual-mode system, supporting both the original 4-bit Vanilla operation and a new 2-bit/4-bit configurable SIMD mode.

2.1 Model Training

To enable 2-bit SIMD execution, we targeted the 27th convolution layer of VGGNet and "squeezed" its dimensions by reducing input/output channels from 64 to 16. We quantized activations to 2 bits while maintaining 4-bit weights. The resulting model achieved an accuracy of 89.46%. Validation was performed by comparing the hardware-simulated `psum_recovered` against the pre-hooked input of the subsequent layer, yielding a reconstruction error of 1.47×10^{-7}.

2.2 Hardware Architecture

To support SIMD execution, the MAC tile and datapath are extended:

- A secondary weight register is added to allow sequential loading of two 4-bit weights; The 4-bit activation value (a_q) is reinterpreted as two packed 2-bit activations, enabling dual parallel MAC operations. The multiplier and adder paths are split to process the upper and lower 2-bit segments independently. Internal multiplexers configure whether results accumulate via standard addition or shifted addition.

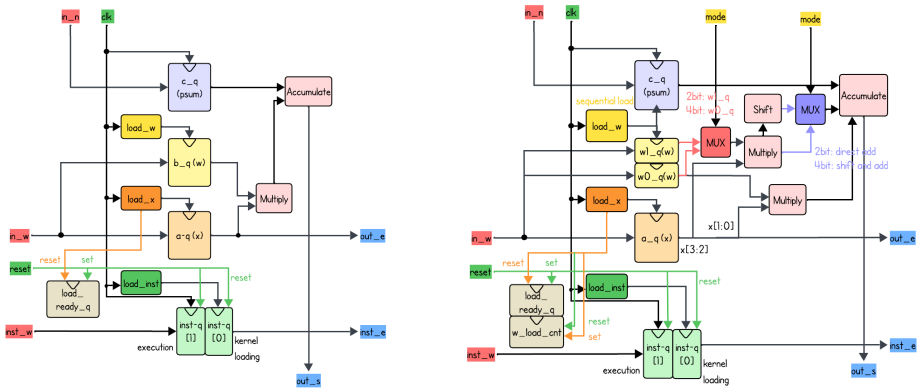


Figure 4: MAC Tile modifications for SIMD configuration

2.3 Testbench and Verification

The overall Core execution flow mirrors the SISD design, but SIMD operation introduces two key changes:

- Weight loading requires 2× the cycles, since two weights must be staged for each MAC tile.
- Execution iterates twice, producing two output streams corresponding to the two 2-bit activation lanes.

Part 3

This part reconfigure the Processing Elements (PEs) and core architecture into an 8×8 output-stationary systolic array.

3.1 Hardware Architecture

Compared to the Weight-Stationary (WS) design, OS mode restructures the MAC stack to support partial-sum retention and output draining:

- MAC Tile: Internal multiplexing is designed to reinterpret instruction modes (execute vs. drain) and to reroute weight, activation, and partial-sum paths accordingly.
- MAC Row: The valid/write-enable logic asserted when the north row drains output instead of execute.
- MAC Array: Output behavior is modified from outputs always come from the last row to once the drain instruction reaches the bottom row, rows output sequentially as partial sums are flushed.

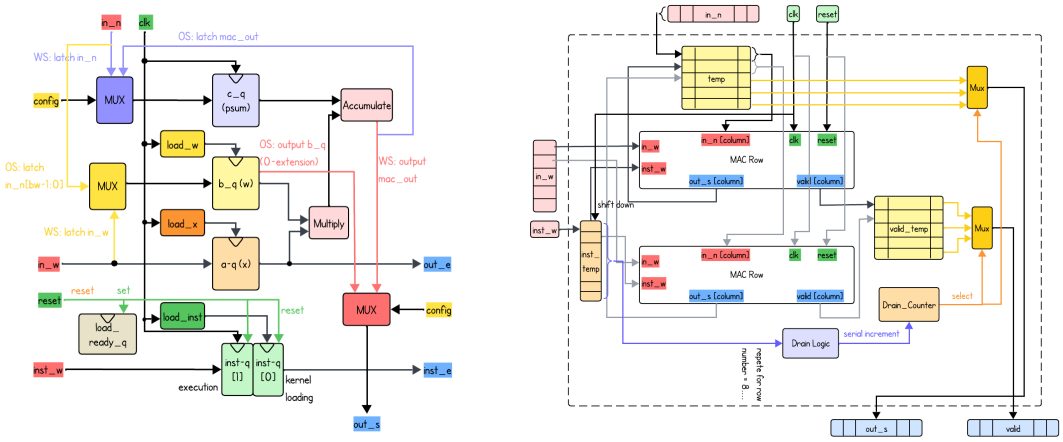


Figure 5: MAC Tile and Array modifications for OS configuration

In addition, an Input FIFO is introduced to stream weights during execution. It mirrors the L0 buffer's row-wise write/read pattern, supporting wavefront generation for the `MAC Array`. Details are provided in Appendix C.1.

3.2 Testbench and Verification

The verification flow parallels the WS methodology. A software model first simulates OS behavior to produce golden traces, a direct comparison between WS and OS instruction loops and dataflow differences can be found in Appendix C.2. The testbench drives the Core through the following sequence in OS mode. Compared with WS mode, the L0 buffer depth is sufficient for a full activation cycle, enabling strictly sequential movement, in OS mode, the combined data requirement for a complete cycle is larger, so the OS design must continuously stream weights and activations from SRAM during execution.

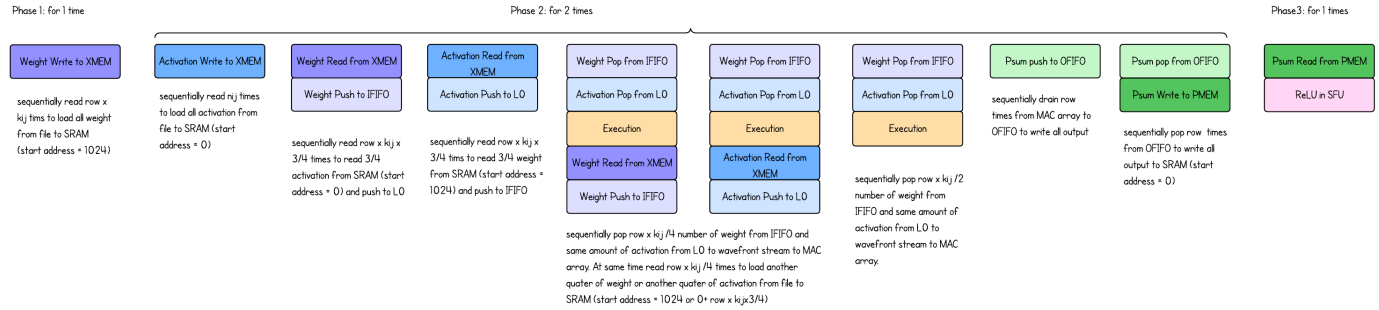


Figure 6: OS Execution Process

Part 4

4.1 Efficient Leaky ReLU Implementation

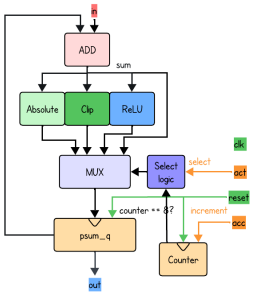
We extend the SFU to support Leaky ReLU using a shift-based implementation, where the slope α is constrained to powers of two ($\alpha = 2^{-\text{act}}$). This avoids explicit multipliers, replacing them with arithmetic right shifts, significantly reducing area and dynamic power. The instruction width is extended from 34 to 37 bits to encode the activation mode and shift amount. All eight configurations were verified with bit-accurate matching; implementation details are provided in Appendix D.1.

4.2 Parameterized Multi-Mode SFU

The SFU is further generalized into a multi-mode activation unit, enabling runtime selection among Linear, ReLU, ReLU6, and Absolute Value via instruction-level control. This allows a single accelerator instance to support diverse network architectures without hardware recompilation.The activation definition and usage cases are shown in table below.

act_mode	Activation Type	Function Definition	Used In
00	Linear	$f(x) = x$	Debug, Quant Sweep
01	ReLU	$f(x) = \max(0, x)$	VGG, ResNet
10	Clipped ReLU (ReLU6)	$f(x) = \min(\max(0, x), 6)$	MobileNet, Low-power
11	Absolute Value	$f(x) = \lvert x \rvert$	—

(a) Multi-Mode Activation



(b) Multi-Mode SFU

We validated the functional correctness of all four activation modes and the FPGA synthesis result along with comparison with Leaky ReLU version is detailed below. Despite added control logic, synthesis results show a 15% improvement in energy efficiency compared to the Leaky ReLU-only baseline, demonstrating effective power optimization through reduced dynamic switching.

Configuration	Max Freq (MHz)	Dyn Power (mW)	GOPS	GOPS/W
Leaky ReLU (Alpha 1)	129.58	41.1	16.59	403.55
Multi-Mode (Alpha 2)	115.33	31.9	14.76	462.76

4.3 Runtime Configurable Structured Pruning

We implemented runtime configurable column gating for structured pruning. This time, we extended instruction from to 38 bits with additional bits `num_active_cols` for dynamic 0-8 column configuration. This design allows the hardware to dynamically disable inactive PE columns, effectively reducing dynamic power consumption ($P_{dyn} = \alpha C V^2 f$) by minimizing the switching activity factor in unused logic.

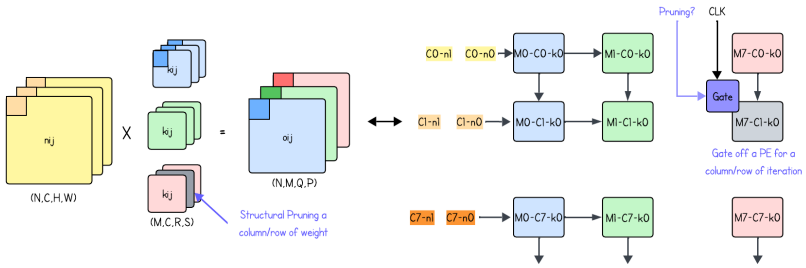


Figure 7: Pruning and Gating architecture

We applied L_2-norm structured pruning to kernel of VGGNet 27th layer and tested three configurations:

Configuration	Active Columns	Pruning Ratio	Parameters Removed
Baseline	8	0%	0
Moderate	4	50%	1,179,648
Aggressive	2	75%	1,769,472

Table 1: Pruning configurations on VGGNet 27th layer.

The synthesis results indicate a trade-off between power adaptability and operating frequency. As shown in the table below (see the visual chart in Appendix D.2), the Gated design incurs a frequency penalty of 15–31% compared to the Baseline, primarily due to the enable comparator ($i <$ num_active_cols) in the critical path and increased routing congestion. However, this reduction in frequency results in lower absolute power consumption: the gated design consumes 17.9% less power than the baseline at full utilisation (8 columns) (27.67 mW vs 33.70 mW). While the baseline is more efficient in terms of GOPS/W (1839 vs. 1550), it exceeds the 20 mW power budget of our target edge devices.

Config	Baseline (mW)	Gated (mW)	Power Savings	Freq Penalty
2 Columns	21.99	19.08	13.2%	-15.3%
4 Columns	25.41	23.19	8.7%	-20.6%
8 Columns	33.70	27.67	17.9%	-30.7%

Table 2: Power-frequency trade-off across configurations.

The energy efficiency analysis in the table below (see the visual chart in Appendix D.3) reveals that energy per operation (pJ/op) increases at lower utilisation (2.5 pJ/op at 8 columns $\rightarrow$ 6 pJ/op at 2 columns) due to the amortisation of fixed overheads. Consequently, column gating should be triggered by strict power constraints rather than for pure efficiency optimisation.

Metric	Baseline (8-col)	Gated (8-col)	Difference
Throughput (GOPS)	61.97	42.89	-30.8%
Power (mW)	33.70	27.67	-17.9%
GOPS/W	1839	1550	-15.7%

Table 3: Full-load comparison: baseline higher efficiency, gated fits power budgets.

Overall, the runtime-configurable pruning implementation achieves a 13–18% power reduction at the cost of a 30% frequency degradation, which validates the use of clock gating for structured pruning support. However, the degradation of energy per operation (pJ/op) at lower utilisation suggests that column gating should be triggered by power constraints rather than efficiency optimization. This transforms a fixed-function accelerator into an adaptive platform, enabling VGG16 inference on edge devices with strict 20–25 mW power budgets.

Metric	Baseline	Gated
Peak Performance	61.97 GOPS @ 120.77 MHz	42.89 GOPS @ 83.71 MHz
Power Range	21.99-33.70 mW	19.08-27.67 mW
Flexibility	Fixed 8-column	Runtime 1-8 columns

Table 4: Trade-off: baseline optimizes throughput/watt (datacenter), gated enables fixed power budgets (edge).

A Part 1: Weight-Stationary Implementation

A.1 VGGNet Training Configuration

Param	Value	Param	Value	Param	Value
Learning Rate	1e-3	Weight Decay	1e-4	Epochs	100
Criterion	CrossEntropy	Optimizer	Adam	Scheduler	CosineAnnealing

A.2 Core Component Architectures

The weight-stationary systolic array system comprises five main components:

1. **MAC Array:** An 8×8 weight-stationary systolic array for matrix multiplication.

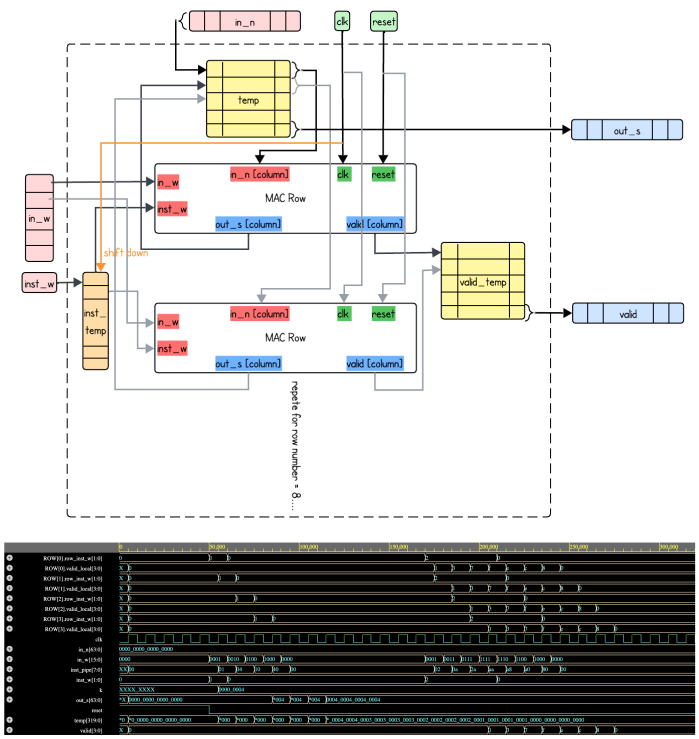


Figure 8: Systolic MAC Array Architecture and functional verification waveform.

2. **L0 Buffer:** Receives weight vectors from SRAM and shifts them row-wise into the MAC array.

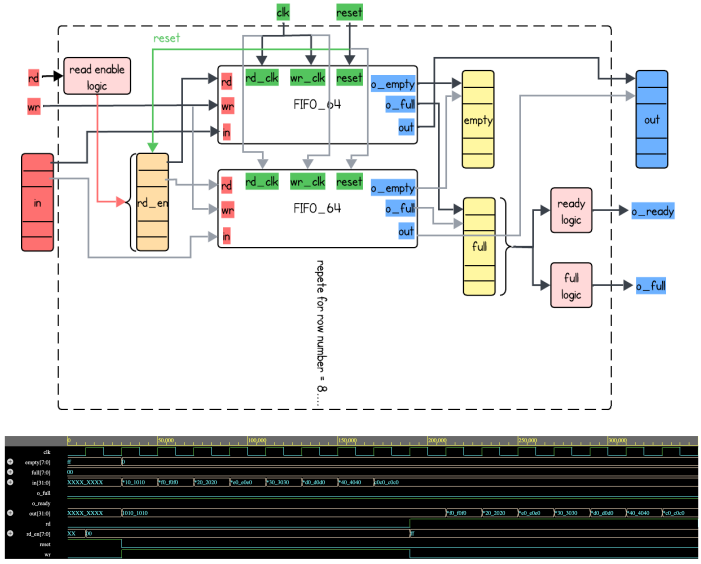


Figure 9: L0 Buffer design and functional verification waveform.

3. **OFIFO:** Output FIFO that gathers 8 columns of partial sums from the array and streams them to psum SRAM.

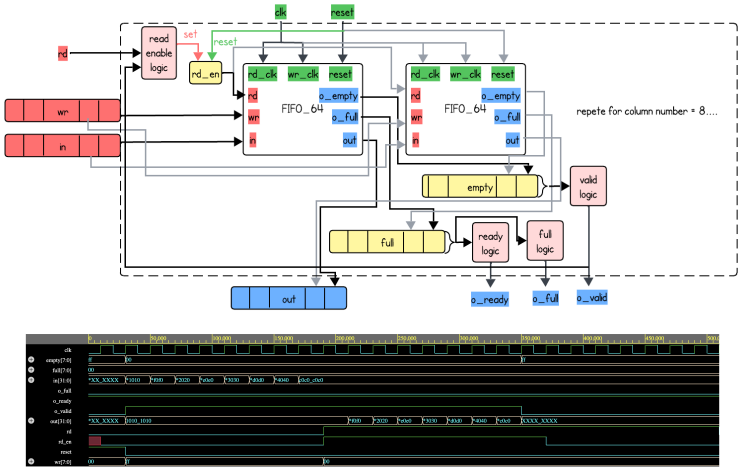


Figure 10: OFIFO architecture and functional verification waveform.

4. **SFU:** Special Function Unit performing accumulation and ReLU activation.

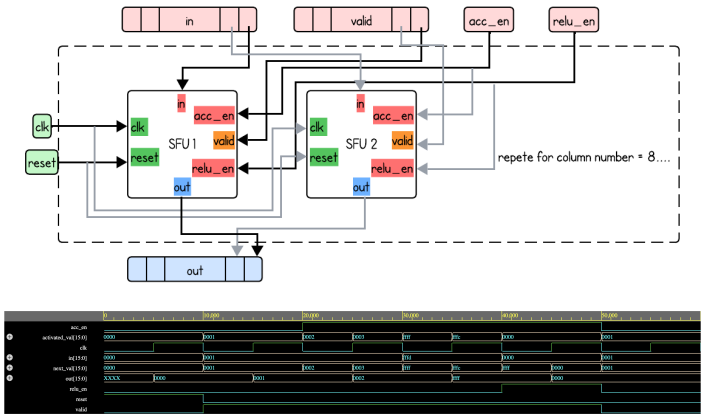


Figure 11: SFU architecture and functional verification waveform.

5. **Scratchpad:** Dual-SRAM configuration (Input SRAM and Psum SRAM) with data widths:

$$\text{Input/Psum SRAM Width} = \text{Rows/Columns} \times \text{ Input/Psum Data Width}$$

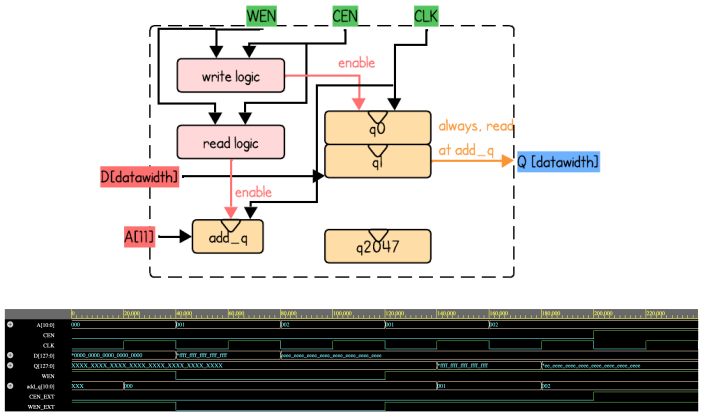


Figure 12: SRAM architecture and functional verification waveform.

A.3 Software Trace Generation

To ensure hardware correctness, we developed a software model that mirrors the hardware datapath. It extracts quantized weights and activations from the trained PyTorch model, performs the convolution operation in software, and emits the expected output traces. These traces serve as the golden reference for verifying the RTL simulation.

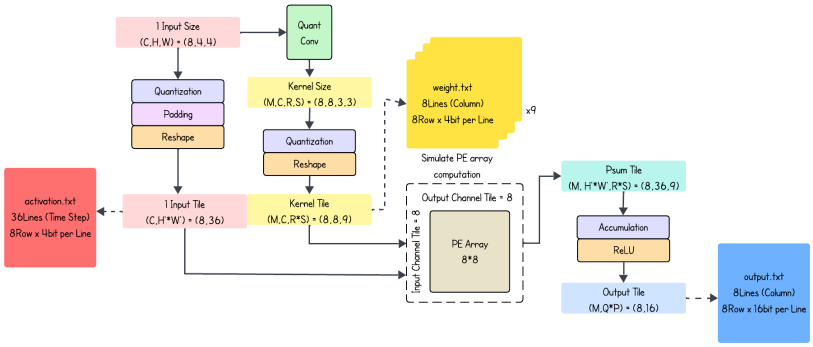


Figure 13: Software trace generation pipeline for hardware verification.

B Part 2: SIMD Configuration

B.1 VGGNet Training Configuration

Param	Value	Param	Value	Param	Value
Learning Rate	1e-3	Weight Decay	5e-3	Epochs	220
Criterion	CrossEntropy	Optimizer	SGD	Scheduler	CosineAnnealing

B.2 FPGA Synthesis Comparison

Table below presents a performance comparison between the baseline Vanilla PE and the SIMD-configurable implementation. While the added multiplexing logic for SIMD support results in a 17% reduction in maximum operating frequency (F_{max}), the architecture gains the capability to process dual 2-bit activation lanes per cycle, potentially doubling throughput for reduced-precision workloads

Metric	Vanilla Version	2-bit/4-bit Configurable
Max Frequency (MHz)	127.89	106.41
Dynamic Power (mW)	31.7	32.1
GOPS	16.37	13.62
GOPS/W	516.4	424.31

C Part 3: Output-Stationary Implementation

C.1 IFIFO Integration

In the Output-Stationary (OS) configuration, weights must be streamed continuously rather than pre-loaded. The Input FIFO (IFIFO) was introduced to facilitate this dataflow. It mirrors the L0 buffer's row-wise write/read functionality, enabling the generation of the diagonal wavefront required for correct systolic execution.

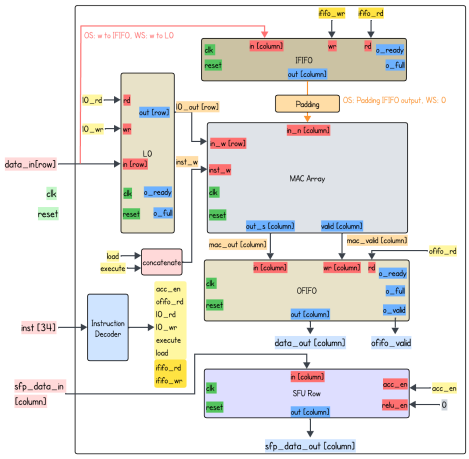


Figure 14: Corelet architecture with IFIFO integration for OS dataflow.

C.2 WS and OS dataflow comparison

The fundamental difference between the two architectures lies in their data retention strategy:

1. Weight Stationary (WS): Kernel weights are pre-loaded into local registers within the MAC array and remain static during execution. Activations are streamed through the array, and partial sums are passed vertically between PEs.
2. Output Stationary (OS): Partial sums are accumulated in local registers within the MAC array and remain static until the calculation is complete. Weights and activations are streamed through the array simultaneously. Once processing finishes, the completed partial sums are drained row-by-row to the OFIFO.

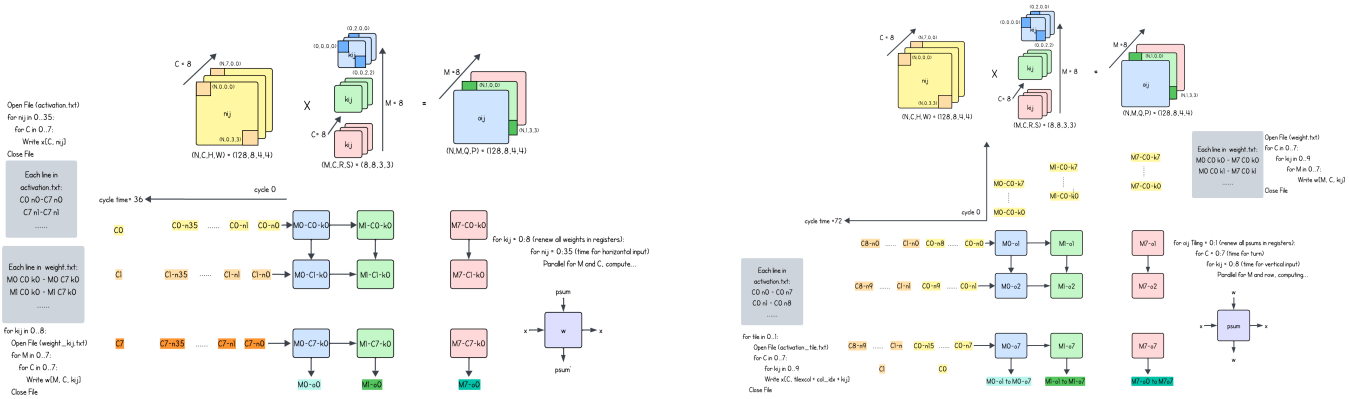
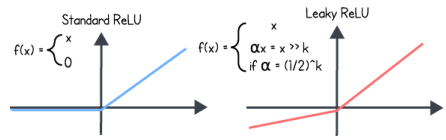


Figure 15: Weight Stationary vs Output Stationary dataflow

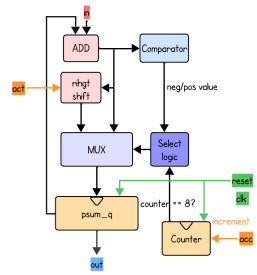
D Part 4: Activation Function Enhancements

D.1 Leaky ReLU Implementation

To address the vanishing gradient problem while minimizing hardware area and power overhead, the Special Function Unit (SFU) was extended to support Leaky ReLU. The hardware architecture, including the modified datapath for shift-based accumulation, is illustrated below.



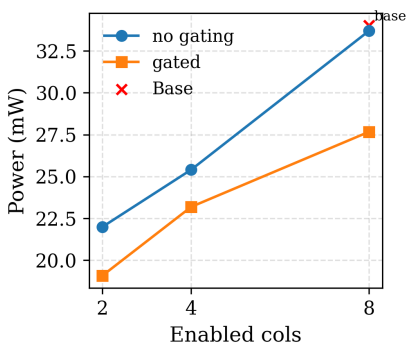
(a) Leaky ReLU Activation



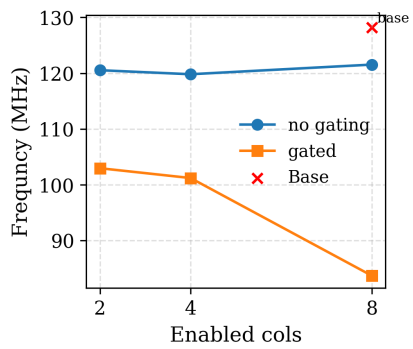
(b) Leaky ReLU SFU

D.2 Power and Frequency Analysis

We analyzed the trade-offs introduced by the runtime configurable column gating. As shown, the gated design consumes 17.9% less power than the baseline (27.67 mW vs. 33.70 mW). However, this flexibility incurs a frequency degradation of 15-31% compared to the baseline.



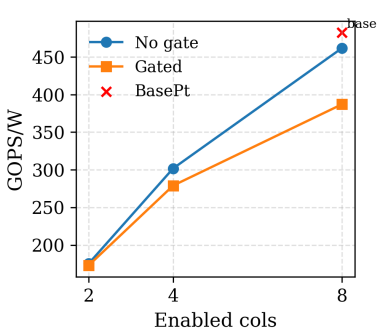
(a) Power vs. Columns



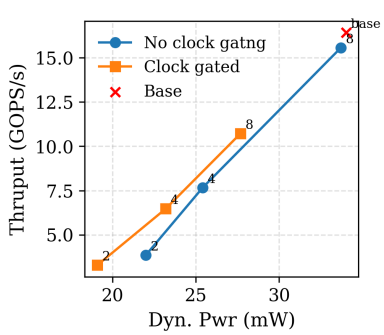
(b) Frequency vs. Columns

D.3 Efficiency and Trade-off Analysis

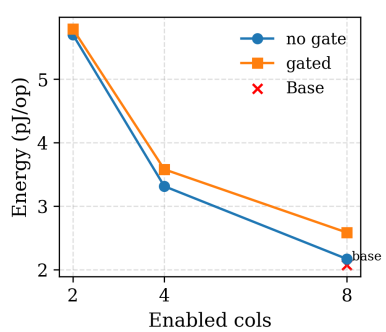
The efficiency analysis reveals a key trade-off between peak performance and operational flexibility. As shown, Baseline has a higher GOPS/W due to frequency. Gated operates at a lower power of 27.67 mW (enabling a 20 mW budget operation) versus 33.70 mW. Energy per operation increases at low utilisation due to fixed overhead (2.5 → 6 pJ per operation from 8 → 2 columns).



(a) GOPS/W



(b) Throughput vs. Power



(c) Energy/Op